# HDGrid: A GPU-Native High-Dimensional Grid Index for Approximate Nearest Neighbor Search with Multiple Range Filters

Zheng Wang
Institute of Computing Technology, Chinese Academy of Sciences
wangzheng2024@ict.ac.cn

Guoxin Kang*
Institute of Computing Technology, Chinese Academy of Sciences
kangguoxin@ict.ac.cn

Yutong Zhou
Institute of Computing Technology, Chinese Academy of Sciences
zhouyutong24s@ict.ac.cn

Lei Wang
Institute of Computing Technology, Chinese Academy of Sciences
wanglei_2011@ict.ac.cn

Wanling Gao
Institute of Computing Technology, Chinese Academy of Sciences
gaowanling@ict.ac.cn

Jianfeng Zhan
Institute of Computing Technology, Chinese Academy of Sciences
zhanjianfeng@ict.ac.cn

## Abstract

How Approximate Nearest Neighbor (ANN) search involving with range-filtering has become a core capability in modern vector databases and AI-driven applications. Real-world scenarios often involve multiple filters, introducing steep challenges by disrupting index integrity and causing exponential search space growth. Existing native indexes either preserve a subset of filtered navigation points, compromising query accuracy by retaining irrelevant points, or restrict support to a single range filter and hence have to employs post-filtering to support multiple filters at the cost of efficiency.

In this paper, we propose HDGrid, an innovative high-dimensional grid index to enable efficient ANN search with multiple range filters. By partitioning the combinatorially large filter space into regular grid cells and leveraging a Morton code scheme to quickly locate the corresponding candidate points, HDGrid significantly reduces the number of irrelevant candidates while maintaining high recall. To further maximize performance, HDGrid compresses the high-dimensional grid structure to reduce memory footprint and utilizes the memory bandwidth of modern GPUs. Experiments demonstrate that HDGrid consistently achieves 10x-1000x performance improvements over state-of-the-art native methods on both synthetic and real datasets involving multiple filters, under filtering ratios varying from 0.01% to 80%.

## Keywords

Range-Filtering Approximate Nearest Neighbor Search; GPU-based index; Product Quantization

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
*Conference'17, Washington, DC, USA*
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

**ACM Reference Format:**
Zheng Wang, Guoxin Kang, Yutong Zhou, Lei Wang, Wanling Gao, and Jianfeng Zhan. 2025. HDGrid: A GPU-Native High-Dimensional Grid Index for Approximate Nearest Neighbor Search with Multiple Range Filters. In . ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

## 1 Introduction

Given the limited expressive capacity of a single dense vector [30], structured attributes with numerical ranges play an important roles in improving retrieval accuracy in modern information retrieval systems. As a result, Approximate Nearest Neighbor (ANN) search with range filters has become its core requirement, motivating new algorithmic and system-level innovations. ANN search increasingly involves with multiple range filters in real-world applications such as retrieval-augmented generation [8, 16], intelligent question answering [12], and e-commerce recommendation [19, 21, 24]. However, multiple range filters pose significant challenges to efficient ANN search.

As the dimensionality increases, spatial locality becomes fragmented since geometrically nearby points are scattered across disjoint index regions, resulting in higher search costs [4]. Moreover, which is the ratio of the number of data points satisfying all filter constraints to the total number of data points in the dataset, decreases exponentially with each additional dimension, making valid candidates increasingly sparse.

Recent studies have proposed more sophisticated filter-aware index structures. There are two major lines of work: the methods designed for single-range filters, e.g., RFANN-Dedicated, DIGRA[15], iRangeGraph [31], RangePQ [32], UNIFY[17] and SERF[34]; and those targeting general filtering conditions (e.g., ACORN[23]). However, the existing techniques, whether based on simple stitching or specialized indexing, fail to efficiently support range-filtering ANN search — where multiple filters interact, selectivity varies across dimensions, and the effective candidate space is extremely sparse.

Recent advances in modern hardware, particularly GPUs, have demonstrated remarkable potential for accelerating vector search. GPU-based indices such as CAGRA [22] achieve performance improvements of over two orders of magnitude compared to state-of-the-art CPU graph indices like HNSW. However, existing range-aware ANN methods such as RangePQ, DIGRA, and iRangeGraph

are fundamentally tree-based, whose irregular traversal and pointer-chasing access patterns lead to severe warp divergence and non-coalesced memory access on GPUs. Moreover, their hierarchical metadata incurs substantial memory overhead, making them difficult to scale within limited GPU memory. As a result, these CPU-native designs cannot be directly deployed on GPUs, motivating the need for a GPU-native range filtering ANN framework that aligns with GPU parallelism and memory access characteristics.

To overcome the limitations of existing methods, We propose a high-dimension grid index structure, named HDGrid. At the coarse cluster level, HDGrid partition the embedding space into a large number of fine-grained grids (even more than the number of points) and maintain a discrete prefix-sum structure to estimate the number of points within any range. This grid representation allows precise, lightweight filtering of irrelevant clusters before visiting their inverted lists. HDGrid propose a coarse-to-fine filtering framework that embeds a grid-based discretization into the standard IVFPQ pipeline, which combines Inverted File (IVF) with Product Quantization (PQ). Additionally, HDGrid further encode grid coordinates as Morton codes, enabling efficient range filtering among candidates inside each cluster. This simple yet effective grid embedding seamlessly integrates into existing Inverted File (IVF)-based indexing, providing a scalable and accurate solution for multiple range-filtered ANN search. To overcome GPU memory bottlenecks, we design a lossless prefix-sum compression scheme tailored for ANN filtering, enabling full in-GPU deployment with up six orders of space reduction.

Our main contributions are summarized as follows.

(1) We propose a high-dimension grid index structure for efficient ANN search under multiple range filters, HDGrid. It partitions the embedding space into a large number of fine-grained grids and utilizes a discrete prefix-sum structure to achieve precise, lightweight filtering of irrelevant clusters.

(2) We introduce a Morton code encoding scheme to accelerate nearest neighbor search within qualified sub-clusters. By mapping high dimensional grids coordinates into a one dimensional sequence while preserving spatial locality, this encoding achieves efficient GPU based candidate localization and adjusts its access strategy according to the filtering ratio.

(3) We propose a GPU-native implementation of HDGrid. To overcome GPU memory bottlenecks and enable full in-GPU deployment, we devise a novel lossless prefix-sum compression scheme that delivers up to six orders of magnitude space reduction, ensuring high-throughput data parallelism and fully exploiting modern GPU resources.

(4) We conduct extensive experiments demonstrating that HDGrid can achieve better efficiency and accuracy than existing native methods. While achieving better query accuracy than competitors, achieving a 20×−1000× speedup in query time across all datasets compared to the best-performing CPU-native SOTA method on each dataset .

## 2 Background and Related Works

We now present the necessary definitions and background, and review related work on both state-of-the-practice and state-of-the-art indexes for range-filtering ANN search.

### 2.1 Preliminaries

*Definition 2.1 (Range-filtering ANN search).* Given a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N} \subseteq \mathbb{R}^D \times \mathbb{R}^l$ consisting of $N$ training points, a query vector $\mathbf{q} \in \mathbb{R}^D$, and a set of range constraints $\mathcal{R} = [\mathbf{L}, \mathbf{U}]$ with $\mathbf{L}, \mathbf{U} \in \mathbb{R}^l$ and $\mathbf{L} \leq \mathbf{U}$ element-wise, forming a hyper-rectangle,

$$\mathcal{H}(\mathcal{R}) = \left\{ y \in \mathbb{R}^l \,\middle|\, L_j \leq y[j] \leq U_j,\ \forall j \in [1, l] \right\}, \tag{1}$$

the goal of ANN search with multiple range filters is to return the top $k$ training points

$$\tilde{\mathcal{N}}_k(\mathbf{q}, \mathcal{R}) \subseteq \{(x_i, y_i) \in \mathcal{D} \mid y_i \in \mathcal{H}(\mathcal{R})\}, \quad |\tilde{\mathcal{N}}_k(\mathbf{q}, \mathcal{R})| = k, \tag{2}$$

such that the returned results are approximately among the closest to $\mathbf{q}$ under $\mathrm{dist}(\cdot, \cdot)$ with high probability, though not guaranteed to be the exact top-$k$.

*Definition 2.2 (Recall in Range-Filtering ANN Search).* Let $\mathcal{N}_k(\mathbf{q}, \mathcal{R})$ denote the exact top-$k$ result set for query vector $\mathbf{q}$ under range constraints $\mathcal{R}$, and let $\tilde{\mathcal{N}}_k(\mathbf{q}, \mathcal{R})$ be the result returned by an approximate method, as defined in Equation (2). The recall of $\tilde{\mathcal{N}}_k(\mathbf{q}, \mathcal{R})$ with respect to $\mathcal{N}_k(\mathbf{q}, \mathcal{R})$ is defined as:

$$\mathrm{Recall}(\mathbf{q}) = \frac{\left| \tilde{\mathcal{N}}_k(\mathbf{q}, \mathcal{R}) \cap \mathcal{N}_k(\mathbf{q}, \mathcal{R}) \right|}{k}.$$

In other words, recall measures the proportion of true nearest neighbors that are restricted to the feasible region $\mathcal{H}(\mathcal{R})$ and successfully retrieved by the approximate algorithm. The overall recall of a method is computed as the average recall over a set of queries.

### 2.2 IVFPQ

IVFPQ [14], short for Inverted File with Product Quantization, is a hybrid algorithm for efficient ANN Search (ANN) search in high-dimensional spaces, widely used in large-scale ANN systems due to its balance of memory efficiency and search speed. It combines Product Quantization (PQ), and Inverted Indexing.

PQ is a widely adopted technique for compressing high-dimensional vectors and enabling efficient ANN search. Given a vector $\mathbf{x} \in \mathbb{R}^D$, PQ partitions the space into $M$ disjoint subspaces of dimension $D/M$, such that:

$$\mathbf{x} = (\mathbf{x}[1], \mathbf{x}[2], \ldots, \mathbf{x}[M]), \quad \mathbf{x}[m] \in \mathbb{R}^{D/M}.$$

For each subspace $m \in [1, M]$, a separate codebook $C[m] = \{\mathbf{c}_1[m], \ldots, \mathbf{c}_K[m]\}$ is trained, typically using $k$-means. The sub-vector $\mathbf{x}[m]$ is then approximated by its nearest centroid:

$$q[m](\mathbf{x}[m]) = \arg \min_{\mathbf{c} \in C[m]} \|\mathbf{x}[m] - \mathbf{c}\|_2^2.$$

The final quantized representation of $\mathbf{x}$ is a compact sequence of centroid indices:

$$\mathrm{PQ}(\mathbf{x}) = [q[1](\mathbf{x}[1]), \ldots, q[M](\mathbf{x}[M])],$$

resulting in a few-byte encoding per vector, which drastically reduces memory usage.

PQ supports fast distance estimation between a query vector $\mathbf{q}$ and a quantized vector $\mathbf{x}$. For each subspace, a lookup table of

distances $\|\mathbf{q}[m]-\mathbf{c}\|^2$ is precomputed for all $\mathbf{c} \in C[m]$. The distance between $\mathbf{q}$ and $\mathbf{x}$ is then approximated as:

$$d(\mathbf{q}, \mathbf{x}) \approx \sum_{m=1}^{M} \|\mathbf{q}[m] - q[m](\mathbf{x}[m])\|_2^2,$$

known as symmetric distance computation (SDC). This avoids explicit vector reconstruction and enables efficient similarity search.

To further improve efficiency, PQ is often combined with inverted indexing [9, 10]. A coarse quantizer partitions the space into $n_{\text{list}}$ clusters, and each database vector is assigned to the closest cluster. At query time, only a small number of nearby clusters are probed, and distances are computed within them using PQ and SDC.

## 2.3 Existing Solutions

There are two general approaches to enabling range-filtering in approximate nearest neighbor (ANN) search. The first is the *stitched* approach, which augments existing ANN indexes with additional pre-filtering or post-filtering components — without modifying the original index structure. Modern vector databases [1, 2, 20, 28, 29, 33] such as Milvus and Weaviate, as well as indexing libraries [3, 7**?** ] like RAFT and Faiss, adopt different stitched strategies: Milvus leverages a bitset mechanism for pre-filtering; Weaviate integrates inverted file structures to support pre-filtering; RAFT employs either bitsets or bitmaps before ANN search; and Faiss supports post-filtering based on metadata stored within the index. Under low filtering ratios, both pre-filtering and post-filtering become ineffective: the former disrupts index structure by aggressively pruning data, while the latter yields insufficient candidates due to sparse filter matches.

The second category is the *native* solution, which integrates range filters directly into the ANN index structure. ACORN [23] attempts to preserve graph connectivity for all potential filtering conditions by increasing the out-degree of nodes in the $k$NN graph, thereby supporting multiple filters simultaneously. iRangeGraph [31] constructs a separate index for each filter and introduces probabilistic traversal to maintain connectivity, even when filtering constraints are applied. DIGRA [15] also builds individual indexes per filter. RangePQ [32] focuses on a single filter by pruning clusters that do not satisfy the given predicate using a PQ-based index. However, these methods face significant limitations under realistic query workloads. ACORN's connectivity degrades significantly when the filtering ratio is extremely low, limiting its effectiveness. iRangeGraph incurs high random access overhead when processing complex multiple filters. Both DIGRA and RangePQ currently support only a single filtering condition, making them unsuitable for scenarios involving multiple filters.

# 3 The HDGrid Solution

We now present an overview of the proposed HDGrid index. Section 3.1 introduces the index structure, which adopts a two-level clustering architecture to improve compression efficiency and facilitate efficient candidate selection, details of which are discussed in section3.2.2 and section3.3.1. Each sub-cluster is equipped with a lightweight multidimensional prefix-sum array that partitions the multi-filter space into $G$ regular grid cells, enabling logarithmic-time pruning of sub-clusters that cannot satisfy all filter constraints.

**Table 1: Summary of Notation**

| Symbol | Description |
|---|---|
| $N$ | the number of training points in datasets |
| $\mathcal{R}$ | input range constraints |
| $k$ | the number of nearest search returned from ANN |
| $D$ | dimension of input vector |
| $l$ | the number of filters |
| $G$ | the numbers of grids |
| $K_1$ | the number of clusters |
| $K_2$ | the number of sub-clusters |
| $S$ | grid resolution |
| $K_1$ | the number of clusters |
| $K_2$ | the number of sub-clusters |
| $n_{\text{probe}}$ | the number of clusters to probe during search |
| $n_{\text{sub_probe}}$ | the number of sub-clusters to probe during search |

Within each sub-cluster, training points are sorted by encoding their filter values using Morton codes, which preserve spatial locality and ensure that all filter dimensions contribute equally—avoiding dominance by any single filter. Section 3.2 describes the index construction process, and Section 3.3 details the query phase. Table 1 summarizes the key notations used.

## 3.1 Index Structure

The proposed HDGrid adopts two-level clustering architecture to group similar training points, each sub-cluster maintains multidimensional prefix-sum arrays to efficiently determines whether it contains any training point satisfying the query's range filters, and applies Morton code encoding to maintain spatial locality among training points sorted by their filter values. Let $\mathcal{D} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_N\}$ denote the set of $N$ vector points, where each point $\mathbf{x} \in \mathbb{R}^d$ represents a $d$-dimensional vector. Each vector $\mathbf{x}$ is partitioned into $M$ disjoint sub-spaces $\{\mathbf{x}[1], \ldots, \mathbf{x}[M]\}$. Within each sub-space $C_j$, we first perform a fine-grained clustering to obtain $K_1 * K_2$ sub-clusters, and then group these sub-clusters into $K_1$ higher-level clusters. This hierarchical clustering architecture maximizes the elimination of sub-clusters irrelevant to a given range query, thereby shrinking the candidate set early.

For each sub-cluster, we define an axis-aligned bounding box in $\mathbb{R}^l$ that tightly encloses all label vectors of the contained points. A prefix-sum array is then built within this bounding box to quickly determine whether the sub-cluster intersects a given query range. The bounding box is discretized into $G$ regular grid cells, where $G$ is typically much larger than the number of training points $n$. Formally,$P = \left\{\prod_{j=1}^{l} [r_j\Delta_j, (r_j+1)\Delta_j) \mid r_j \in \mathbb{Z}\right\}$, where $L_j$ denotes the range of the $j$-th filtering dimension, and $\Delta_j$ denotes the span of each grid at the $j$-th filtering dimension. Given a query vector $q$ with range filters $\mathcal{R}$, HDGrid first determines in $O(1)$ time which grid cells intersect with the query region $\mathcal{R}$, and then uses the prefix-sum array to compute how many training points are contained in these intersecting cells.

Although a query may slightly overlap grids beyond its actual range, the grid granularity is sufficiently fine so that the boundary

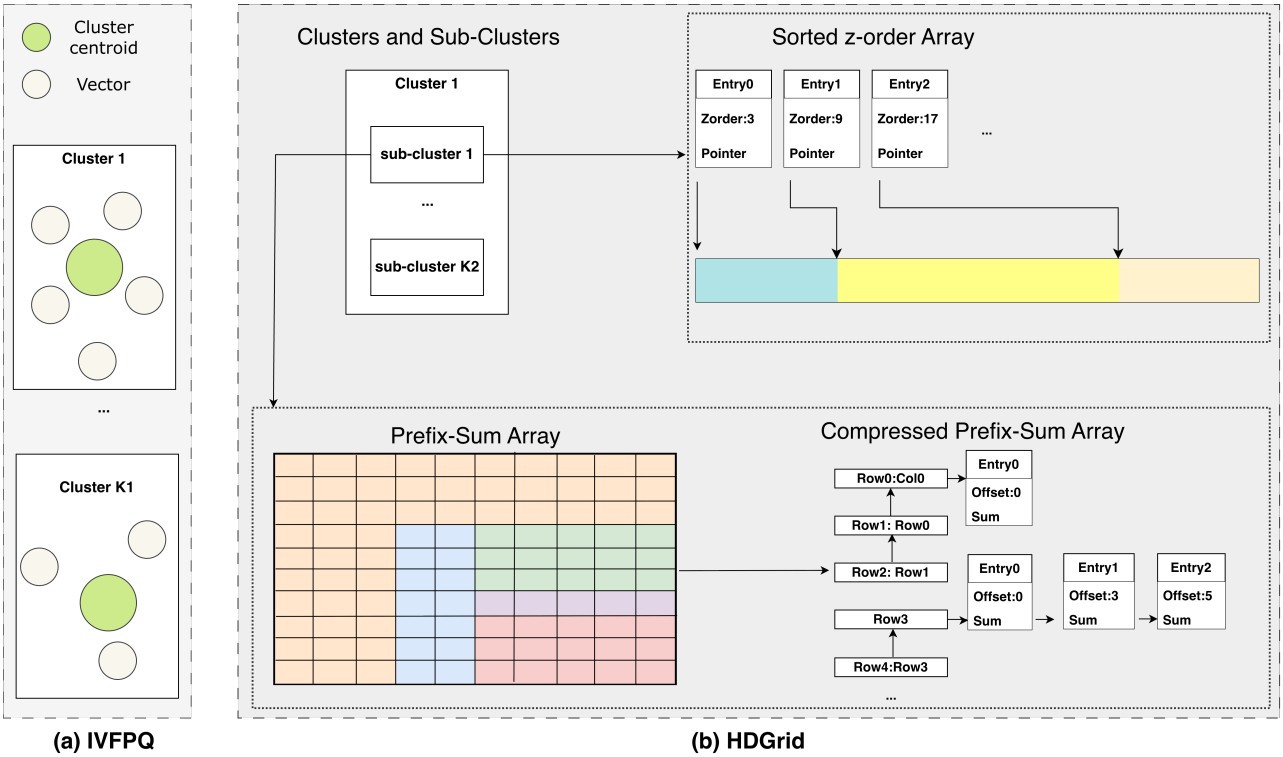


**Figure 1: (a) IVFPQ vs. (b) HDGrid. IVFPQ clusters vectors and stores coarse partitions only for ANN search. HDGrid extends IVFPQ by supporting multiple range filters by introducing a fine-to-coarse cluster hierarchy and equipping each sub-cluster with a compressed prefix-sum and Morton codes arrays, enabling early pruning and minimizing access to irrelevant points.**

overhead is negligible. To further enhance data locality within each sub-cluster, HDGrid encodes the multidimensional filter values of every training point into a one-dimensional Morton code. The Morton encoding orders the multiple filters within each sub-cluster so that points with similar filter values are stored adjacently.

## 3.2 Index Construction

The key design of HDGrid index construction is to achieve fine-grained grouping of similar training points via two-level clustering, and attach each sub-cluster with a prefix-sum array for filter-aware pruning and Morton-code sorting for locality-preserving access. Both of these steps are summarized in Fig. 1.

*3.2.1 Building two-level clustering.* Alg. 1 illustrates the construction of a two-level clustering that partitions the training data into fine-grained, semantically coherent groups. We first use K-means[18] algorithm to divide the training points into $K_1 \times K_2$ sub-clusters using the K-means[18] algorithm. These sub-cluster centroids are then grouped into $K_1$ coarse clusters via a second-level clustering step. This fine-to-coarse partitioning strategy captures local structures before global aggregation, effectively minimizing intra-cluster variance and improving overall clustering cohesion.

*3.2.2 Building a Compressed High-Dimensional Prefix-Sum Array.* Then, for each sub-cluster, we construct a multidimensional prefix-sum array to efficiently determine whether it satisfies the query's range filters. The $l$-dimensional axis-aligned bounding box in $\mathbb{R}^l$ that tightly encloses all label vectors of the contained points is uniformly partitioned into $G$ regular grid cells. The number of points contained in each grid cell is computed in Alg. 2, where a binary array marks the presence of training points in grid cells, and a prefix sum over this array computes the write offsets. These offsets indicate where valid results should be written in memory, ensuring efficient and contention-free parallel writes.

All loops in the algorithm are fully parallelizable, as their iterations are independent. The routines GetGridCoords and FlattenCoords map each point to its corresponding grid cell and then to a one-dimensional index using row-major order. A high-dimensional prefix sum is finally computed using a standard dimension-wise scan [26].

**Algorithm 1:** Fine-to-coarse Clustering Construction

**Input:** Dataset $\mathcal{X} = \{x_i\}_{i=1}^N$, $x_i \in \mathbb{R}^d$, number of clusters $K_1$, sub-clusters per cluster $K2$

**Output:** Inverted lists ivf; Coarse cluster centers $\in \mathbb{R}^{d \times K_1}$; subcenters $\in \mathbb{R}^{d \times (K_1 \cdot K2)}$

1 ivf, subcenters $\leftarrow$ kmeans$(\mathcal{X}, K_1 \cdot K_2)$ ;

2 centers $\leftarrow$ kmeans_balanced(subcenters, $K_1$) ;

**Algorithm 2:** GridsCounting

```
Input: Attributes y ∈ ℝ^{l×N}, Grid Resolution S ∈ ℝ^l
Output: Number of Points in each Grids
        Count[S_1 × S_2 × .... × S_l]
gridCoords ← GetGridCoords(S, y) ;
flatCoords ← FlattenCoords(S, gridCoords) ;
sortedFlatCoords ← sort(flatCoords) ;
diffLabel[1] = 1 ;
for i = 2 to N do
    if sortedFlatCoords[i] ≠ sortedFlatCoords[i − 1] then
        diffLabel[i] = 1 ;
    end
end
diffLabelSum = SumScan(diffLabel) ;
for i = 2 to N do
    if diffLabel[i] = 1 then
        counts[diffLabelSum[i]] ← i ;
        gridId[diffLabelSum[i]] ← sortedFlatCoords ;
    end
    if i = N then
        uniqueCount = diffLabelSum[i] ;
    end
end
for i = 1 to uniqueCount do
    Count[gridId[i]] = counts[i] ;
end
```

Because the prefix-sum array stores one entry per grid cell, its space cost can be substantial—e.g., 30 million entries in our experimental setup, Storing all such arrays for million-scale datasets would require more than 50 GB of memory, exceeding the capacity of many high-end GPUs (e.g., NVIDIA V100). Therefore, we compress them by exploiting redundancy, as shown in Alg. 3. The core idea is to store only the starting offset and representative value for each run of identical prefix sums within a row. Moreover, we observe significant duplication across rows: many are exact copies of previous ones. In such cases, as handled in Alg. 2, we store only an offset pointing to the last valid row. This optimization reduces storage and enables fast reconstruction. The theory 1 provides lower and upper bounds on the achievable compression rate.

Theory 1. *The space complexity for the compressed prefix-sum array is between* $\theta(\frac{GK_1K_2}{S[l]})$ *and* $\theta(\frac{NG}{S[l]})$

Theory 2. *The expected space complexity for the compressed prefix-sum array is* $o(\sqrt{N}G \max\left(1 - \exp\left(-\frac{N}{K_1K_2\sqrt{G}}\right), \frac{1}{M^{\frac{1}{2(l-1)}}}\right))$.

Theory 1 gives an important property of the compress algorithm that the compress ratio is highly dependent on the grid step of $lth$ dimension, the higher $h[l]$ gives higher compress ratio. However, gives total number of grids $G$, given $lth$ dimension higher resolution means other dimension share few grids, witch would influence query performance as will be discussed in section 3.3. To trade-off performance and memory consumption, we choose $S[l] = \sqrt{G}$ and

**Algorithm 3:** CompressPrefixSumArray

```
Input: Grid Resolution S ∈ ℝ^l, the l dimensional prefix sum
       array P[S_1 × S_2 × .... × S_l], number of grids G
Output: Last valid sum node array lastArray, array of sum
        nodes saving summation value sumNodes
mainLen ← S[l] ;
reaminLen ← G/S[l] ;
for i = 1 to remainLen do
    if mod(i, S[l − 1]) = 0 then
        rowMask[i] ← 1 ;
        continue ;
    end
    rowMask[i] ← 0 for j = 1 to mainLen do
        flatten ← i ∗ mainLen + j ;
        if P[flatten] ≠ P[flatten − 1] then
            rowMask[i] ← 1 ;
            break ;
        end
    end
end
for i = 1 to remainLen do
    nodes ← ∅ ;
    lastArray[i] ← last ;
    lenArray[i] ← 0 ;
    if rowMask[i] = 1 then
        last ← i ;
        for j = 1 to mainLen do
            flatten ← i ∗ mainLen + j ;
            if j = 1 or P[flatten] ≠ P[flatten − 1] then
                Appen(nodes, (j, P[flatten])) ;
            end
        end
        sumNodes[i] ← nodes ;
    end
end
```

remaining dimension with equally resolution $M^{\frac{1}{2(l-1)}}$. Crucially, Theory 2 not only provides a practical compression rate for this configuration but also reveals a significant increasing trend in the compression ratio relative to the total number of inverted lists. This finding is further and robustly validated in our experimental evaluation. This compress schema is loss-less, and data in the prefix-sum array can be retrieved with time complexity $O(2^l log(N))$ without decompress the array at run time.

*3.2.3 Building the Sorted Morton-code Array.* This fine-to-coarse organization in Sec. 1 provides a structured foundation for efficient data placement and subsequent Morton-based ordering. Within each sub-cluster, the training points are sorted according to their Morton code values to enable fast retrieval. As illustrated in Fig. 1, all data identifiers in a sub-cluster are grouped into multiple arrays based on the Morton code values derived from their grid coordinates. In practice, HDGrid stores all data identifiers and their associated

grid coordinates, encoded using Morton codes, contiguously in memory and then sorts them by Morton code value. As a result, records sharing similar attribute values are automatically collocated in memory. This design is conceptually simple, highly parallelizable, and exhibits strong memory locality, making it particularly suitable for GPU implementations. Furthermore, it establishes a foundation for the performance optimizations discussed in Sec. 3.3.

## 3.3 Query Processing

Given a query vector $q$ and its range constraints $\mathbb{R}^l$, the query process in HDGrid aims to efficiently identify training points that meet all specified filtering conditions. The index first selects the coarse clusters that satisfy the given range constraints, and then further refines the search by using the prefix sum array to determine the sub-clusters that also meet these constraints. Finally, within each qualified sub-cluster, the Morton code based mechanism precisely locates the candidate region, enabling efficient retrieval of the nearest neighbors.

*3.3.1 Select Candidate Sub-clusters.* The first task of search phase in HDGrid is selecting the $n_{sub_probe}$ nearest sub-cluster conatins data within the query, as shown on line 1-5 of algorithm 3. The key routine of this process is $QueryPreifxSum$ in line 1, where number of points within the query in each of sub-clusters is retrieved from the compressed high dimensional prefix sum array. This procedure extends the classical high-dimensional prefix sum query algorithm by leveraging the principle of inclusion-exclusion, despite the summation value is not retrieved from the array directly, but is searched in our compressed prefix sum, as shown in Algorithm 4:

As discussed in section 3.2.1, in our compressed prefix sum array, each sum nodes is a 2-tuple with first element indicate the offset and second indicate the sum value. The sums within each node are stored in increasing order of their corresponding offsets, the last summation with offset smaller than the last dimension of $gridCoord$ is the expected result. This can either be get through a binary search or a linear search. Since binary search involve more random access, this is inefficient especially in GPU environment. Therefore, a thresh hold length is set to decides whether to use linear search or binary search.

**Algorithm 4:** QueryPrefixSum

**Input:** Last valid sum node array $lastArray$, array of sum nodes array saving summation value $sumNodes$, A thresh-hold value $threshHold$, Range bounds $\mathbf{L}, \mathbf{U} \in \mathbb{R}^l$, Grid Resolution $\mathbf{S} \in \mathbb{R}^l$

**Output:** number of points Within the filter in current sub-cluster $sum$

```
for i = 0 to 2^l do
    sign ← 1 ;
    for j = 0 to l do
        bit ← GetBit(mask, i) ;
        if bit = 1 then
            sign* = −1 ;
            coords[j] ← U[i] ;
        end
        else
            coords[j] ← L[i] ;
        end
    end
    gridCoord ← GetGridCoords(coords, S) ;
    lastLen ← mod(gridCoord, S[l]) ;
    remain ← floor(gridCoord/S[l]) ;
    node ← sumNodes[remain] ;
    if node = ∅ then
        node ← sumNodes[lastArray[remain]] ;
    end
    if len(node) > threshHold then
        sum ← BinarySearch(node, lastLen) ;
    end
    else
        sum ← LinearSearch(node, lastLen) ;
    end
end
sum* = sign ;
```

After get number of points in each sub-cluster of current query, $GetClusterCounts$ simply sum number of points in all sub-cluster of each coarse cluster to get number points within each cluster of current filter. Then $FilterNoneZero$ filtered coarse clusters witch not contain any points of current query, this routine is implemented efficiently by classic stream compaction[6] on GPU. Following this the nearest $n_{probe}$ coarse cluster are selected from filtered clusters. Similar process is applied to all sub-clusters to find the k nearest sub-clusters.

Discussion of performance: The total time complexity for this stages (Algorithms **??** and **??**) comprises two main components. The first component, which involves , is approximately $O(K_1K_2 2^l \log(\frac{N}{K_1K_2}))$. The second component, which accounts for calculating the distance between the query and candidate sub-clusters and selecting the nearest ones, is given by $O(n_{probe}K_2D + \log(n_{probe}K_2)n_{sub_probe})$ if using a GPU priority queue-based implementation. If we were to use a single-stage clustering structure with $K_1K_2$ clusters directly, the full time complexity would be $O(K_1K_2D + \log(n_{probe}K_2)n_{sub_probe} + \log(n_{probe}K_2)n_{probe})$. By comparing the complexity terms, the primary performance gain of the two-stage strategy stems from replacing the term $O(K_1K_2D)$ with $O((n_{probe}K_2 + K_1)D)$ for initial cluster distance computations. In typical **Inverted File with Product Quantization (IVFPQ)** setups, the number of clusters ($K_1$) is often set around $\sqrt{N}$. Given that $n_{probe}$ is generally a small constant relative to $K_1$, and the number of sub-clusters ($K_2$) is typically between 5 and 10, the two-level clustering approach can decrease the total number of distance calculations between the query and cluster centers by up to **one order of magnitude** at the cost of increasing the total number of clusters by only 5 to 10 times.

Another term will be influenced by hierarchy two-level clustering is $O(K_1K_2 2^l log(\frac{N}{K_1K_2}))$, which is the time for finding the numbers of points within each clusters for given query. However, when l is small (less or equal to 5), this overhead is much smaller than compute accurate distance between cluster centers and queries especially for high dimensional vectors (>=128), which will be evaluated at experiment part. Noted this this term can be further optimized

**Algorithm 5:** SearchMortonLinear

**Input:** Range bounds $\mathbf{L}, \mathbf{U} \in \mathbb{R}^l$, A sorted morton code array $A$
**Output:** Points in $A$ within the rectangle *candies*

```
for i = 1 to l do
    start[i] ← L[i] ;
    end[i] ← U[i] ;
end
left ← MortonEncode(start) ;
right ← MortonEncode(end) ;
l ← LowerBound(A, left) ;
r ← UpperBound(A, right) ;
candies ← FilterInRectange(A, rec, l, r) ;
```

to $O(K_1 2^l log(\frac{N}{K_1 K_2}) + n_{probe} * K_2)$ by build separate prefix-sum array for each clusters. However, this would double the memory overhead witch would cause finer dataset partition, and influence the final throughput. We recommend using this optimization only for small dataset.

*3.3.2 Retrieve Candidate Data Identifiers.* After determining the candidate sub-cluster, the next task is to retrieving data identifiers from these sub-cluster. All data identifiers are grouped by the grids they belonging to, and are sorted based on the morton-code of their grid coordinate. Each filter in multiple range filtered ANN search is hyperrectangle in $\mathbb{R}^l$, it will be transformed to grid coordinate in whitch is a hyperrectangle in $\mathbb{Z}^l$. Then retrieving data identifier within the filter is equivalent to visit all valid grids within the hyperrecatangle. This classic computational geometry can be solved with time complexity $O(log^{l-1} + k)$ [4], where k is the number points within hyperrectangle. However, these solution always involve high space complexity rooted in classic tree-based structures like K-D trees[5] and R-trees [11]. To trade space complexity and performance, HDGrid supplement two strategies for this based on morton-code. The first one simply transform the hyperrectangle to a morton-code range using the property of morton code:

Lemma 3.1.

$$\forall \mathbf{x}, \mathbf{y} \in \mathbb{Z}^l, \bigwedge_{Z_i^t \in \mathbb{Z}^l} x_i \leq y_i \rightarrow morton(\mathbf{x}) \leq morton(\mathbf{y})$$

then traverse this morton-code range and filtered points not in the hyper-rectangle, as shown in Algorithm 5:

The SearchMortonLinear routine in Algortithm 5 may visit some out of rectangle points, but since memory access pattern exhibits strong spatial locality and the sub-procedure *FilterInRectange* can be implemented with stream compaction[6] efficiently on GPU, we find it is enough for low ( $\leq 3$) dimensional attributes with medium filter ratio (higher than 1%), witch do not become performance bottle-neck shown in our experiment part. But for attribute with higher dimension with extremely low filter ratio, the ratio of points out of hypperrectangle exponentially increase, the algorithm may degenerate to linear scan all points within each sub-cluster. Therefore, we propose the second strategy, in witch the morton code range of hyperrectangle is split into multiple partitions, only

**Algorithm 6:** SearchMortonWithSplits

**Input:** Range bounds $\mathbf{L}, \mathbf{U}$, A sorted morton code array $A$, numbers of splits $s$
**Output:** Points in $A$ within the rectangle *candies*

```
for i = 1 to l do
    start[i] ← L[i] ;
    end[i] ← U[i] ;
end
left ← MortonEncode(start) ;
right ← MortonEncode(end) ;
len ← (right − left)/s ;
for i = 1 to s do
    l ← LowerBound(A, left + (i − 1) ∗ len) ;
    r ← UpperBound(A, left + i ∗ len) ;
    isValid ← true;
    if InRec(A[l], rec)orJumpIn(A[l], rec)) ≤ r) then
        splitCandies ← FilterInRectange(A, rec, l, r) ;
        Append(candies, splitCandies) ;
    end
end
```

partitions contains points within the hypperrectangle are saved, as shown in Algorithm 6:

The sub-routine *JumpIn* find the smallest morton-code value in the hypperrectangel that is larger than a given value. As the principles of the *JumpIn* algorithm are well-documented in seminal works on spatial databases [13, 25], we omit a detailed redescription here and refer the interested reader to those sources for implementation specifics. The time complexity of JumpIn routine is $O(bits * D)$, where bits denote the number binary bits needed to represent any dimension, both bits and $D$ in realty is a very small constant compared to $N$, so this routine can be implemented vert efficiently.

The reason why we use a split based search algorithm rather than directly using *JumpIn* and *JumpOut*[27] to find a segments within the range in the hypperrectangle is such process is sequential, even it can be paralleled for multiple sub-cluster candidates and still cause high workload imbalance and is inefficient on GPU. In practice, since the morton code array is sparse so only one *JumpIn* call may return a points in the hyperrectangle but not appears in the morton-code array of the sub-clusters, our implementation add new hyper-parameter to control the maximum number of *JumpIn* call to decide whether to save a segments.

The *SearchMorton* routine simply call one of previously discussed search strategy on each sub-cluster and retrieve the corresponding data identifiers at the same time. We omit implementation details here, as the approach is straightforward.

Discussion of performance: Although morton encoding (morton-code) does not have a closed-form analytical guarantee on the expected performance gain, its bit-interleaving property offers several practical advantages over simple row- or column-major layouts. First, ZCode treats all dimensions uniformly, avoiding any bias toward a particular axis. As a result, index construction does not require prior knowledge of query distributions or filtering predicates.

**Table 2: Characteristics of the datasets from BigVectorBench.**

| Dataset | Dim. | Size | #Filters | Type | Filter Ratio | Dynamic |
|---|---|---|---|---|---|---|
| ag_news | 384 | 120k | 1 | text | 25% | not |
| cc_news | 384 | 640k | 1 | text | 10% | yes |
| app_review | 384 | 270k | 3 | text | 1% | yes |
| amazon | 384 | 15900k | 5 | text | 0.01% | yes |
| msong | 420 | 990k | 1 | text | 80% | not |
| deep1m | 256 | 990k | 2 | image | 50% | not |
| gist | 960 | 990k | 3 | image | 0.1% | yes |
| sift5m | 128 | 4990k | 3 | image | 3% | yes |

Second, the balanced spatial locality provided by ZCode naturally leads to more uniform workload distribution when queries are batch-executed on GPUs, which aligns well with our parallel processing model. Third, compared to more sophisticated space-filling curves (e.g., Hilbert or Gray codes), ZCode achieves higher encoding and lookup throughput while still supporting efficient "JumpIn" strategies for range traversal. Considering these properties, we adopt ZCode as our primary encoding scheme in this work.

The expected time complexity in this query phase using the first search strategy is: $O(\mathbb{H}_{linear}(fratio)\frac{Nn_{probe}}{K_1K_2})$, $\mathbb{H}_{linear}(fratio)$ denote a ratio can be avoided to be visited related to real filter ratio of current query, up to our knowledge, it has no closed-form analytical representation and will be evaluated on experiment part. The second strategy needs additional JumpIn call and can filterd out more irrelevant points. The theory bellow tight upper bound for maximum number of JumpIn call:

Theory 3. *The maximum numbers of JumpIn call in 6 is $\theta(\mathcal{A}+s)$ where $\mathcal{A}(\mathcal{R}) = 2\sum_{i=1}^{l}\left(\prod_{j=1,j\neq i}^{l} S_j\right)$ denote the surface area of the filter $\mathcal{R}$, s denote the numbers of splits.*

Then the time complexity of the second strategy is $\theta((\mathcal{A}+s)n_{sub_probe} + \mathbb{H}_{split}(fratio)\frac{Nn_{probe}}{K_1K_2})$, $\mathbb{H}_{split}(fratio)$ will also be evaluated on the experiment part.

*3.3.3 Search k-nearest neighbors.* Following the candidate selection, we perform a Product Quantization-based Approximate Nearest Neighbor (ANN) search on the resulting candidate set to identify the final results. The implementation details are omitted here as they align with standard practices for Product Quantization-based search. Consistent with most PQ methods, our approach includes a final refinement step to improve search recall.

# 4 Evaluation

## 4.1 Experimental Settings

All experiments are conducted on a server equipped with **dual Intel Xeon Gold 6330 @ 2.00GHz CPUs**, **512 GB RAM**, and **four NVIDIA V100 GPUs (32 GB each)**, running **Ubuntu 22.04**. All GPU-based methods are executed on **all four GPUs** in parallel, while CPU-based methods are compiled with **GCC 12.3.0** using the `-O3` optimization flag and executed on a **single thread**, consistent with the setup used in SOTA native methods [15, 31, 32]. We ensured identical software environments (CUDA version, driver, and BLAS backend) across all GPU baselines to avoid bias introduced by hardware or library differences.

*Baselines.* We compare our method with multiple baselines representing the current state of the art for both CPU and GPU implementations. For GPU-based methods, we adopt the **IVFPQ** and **CAGRA** implementations from **NVIDIA RAFT**, and extend them with a *post-filtering stage* to support filtered retrieval. Specifically, we enlarge the number of retrieved candidates to $k \times \beta$, where $\beta$ equals the inverse of the dataset's average filtering ratio, and then apply post-filtering to retain only candidates satisfying all filter predicates. All GPU-based approaches considered in our experiments employ full data residency in Video Random Access Memory, performing the entire Approximate Nearest Neighbor (ANN) search and retrieval workflow exclusively on the GPU.

For CPU-based baselines, we use the **official open-source implementations** of the corresponding systems. For methods that build indexes over a single filter condition (e.g., **DiGRA**, **RangePQ**), which do not natively support multi-attribute filtering, we build the index on the *first filter attribute* of each dataset, and perform post-filtering on the remaining predicates after retrieval.

*Parameter Settings.* For **graph-based methods**, we follow the recommended configurations from prior work and the official repositories: for **HNSW**, the graph degree is fixed to $M = 256$, the construction parameter $ef_c = 300$, and the search parameter $ef_s$ is varied from 15 to 1300. For **CAGRA**, the graph degree is fixed to 64.

For **PQ-based methods**, the number of inverted lists is set to $\sqrt{N}$, where $N$ denotes the dataset size. For datasets with dimensionality below 384, we use $m = 4$ subspaces; for higher-dimensional datasets ($\geq 384$), we set $m = 8$. The number of bits per sub-codebook is fixed at 8 across all datasets.

*Benchmarks and workloads.* We evaluate our method using the workloads selected from BigVectorBench [**?** ], which is currently the only public benchmark that contains realistic filtered vector search workloads. From this benchmark, we select eight representative datasets covering both textual and visual embeddings, diverse filtering ratios, and varying numbers of filtering predicates. These datasets represent a comprehensive mixture of dense and sparse filtering conditions, as well as both static and dynamic filters(where static filters maintain a fixed query selectivity over the workload, while dynamic filters exhibit varying query selectivities). The detailed characteristics of the selected datasets are summarized in Table 2. as $FR(r)$, can be formally expressed as:

## 4.2 Self-evaluation of HDGrid

To better understand the contribution of each component in HDGrid, we conduct a detailed ablation study that highlights the impact of key design decisions. First, we evaluate how Morton-order indexing reduces unnecessary point accesses during the scan phase. Second, we evaluate the effectiveness of the prefix-sum compression scheme in lowering memory usage across various datasets. Finally, we evaluate the additional computational overhead introduced by the two-level clustering structure and the prefix-sum mechanism.

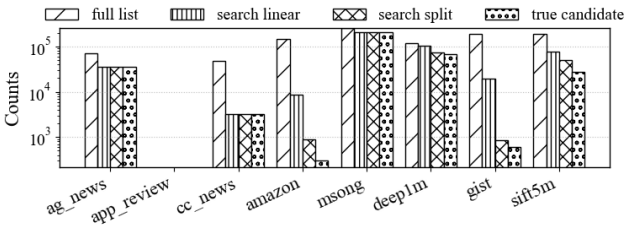


**Figure 2: Effective of the Morton Code Method**

*4.2.1 Effect of Morton-code Search Strategy.* To evaluate the impact of Morton-code strategy on reducing irrelevant point accesses during the scan phase, we conduct experiments across all datasets under recall levels above 90%. For each dataset, we measure four quantities after the coarse clustering stage: (1) the total number of candidate points in the selected clusters, (2) the number of remaining points after applying the *linear search* strategy based on Morton order, (3) the number of remaining points after applying the *split search* strategy, and (4) the number of points that actually satisfy all filtering conditions. The results are shown in Figure 2.

The results demonstrate that both Morton-based strategies significantly reduce the number of irrelevant points visited during scanning. Between the two, the *split search* strategy consistently achieves a larger reduction across all datasets. When the dataset contains one-dimensional labels, the Morton code degenerates to a simple sorted order of labels, allowing perfect identification of points satisfying the filter conditions. For multi-dimensional label spaces, although the effectiveness of the linear search strategy decreases due to less precise spatial locality, it still provides substantial filtering benefits under low filtering ratios. Overall, the split search method reduces irrelevant accesses to less than 5× of the number of true matches on all datasets, effectively suppressing the overhead of scanning in high-dimensional, low-selectivity scenarios.

The improved selectivity brought by Morton-code ordering enables the search to exploit strong spatial locality, thus minimizing unnecessary memory accesses during the scan stage. Although the split strategy introduces a slight computational overhead (less than 1 ms per 100 queries), its effectiveness in reducing unproductive point visits often outweighs the additional cost, especially in workloads with high-dimensional labels and low filtering ratios.

Therefore ,we recommend using the *split search* strategy when the filtering ratio is below 80% or when the dataset involves multiple label dimensions. For simpler, high-selectivity workloads, the linear Morton search provides a lightweight yet effective alternative.

*4.2.2 Effectiveness of the Prefix-Sum Compression Algorithm.* We evaluate the effectiveness of our proposed prefix-sum compression algorithm on eight datasets by measuring the memory consumption before and after compression. To ensure fairness, we keep the clustering and search parameters unchanged and only toggle the compression algorithm.

*Results.* As shown in Fig. 3, our compression algorithm achieves a remarkable compression ratio ranging from 50× up to $10^6$× across all datasets. When the label dimension is one, the algorithm

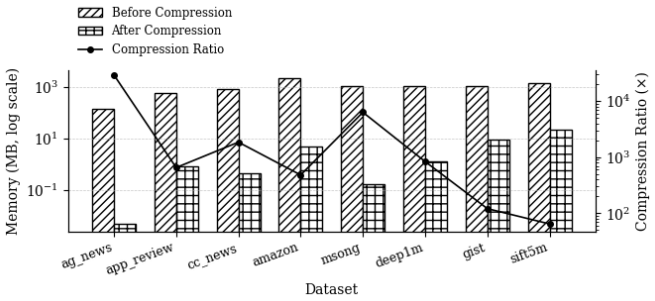


**Figure 3: Ratio-of Compress for Compression Algorithm**

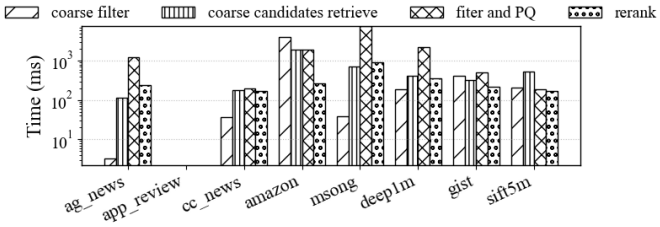


**Figure 4: Performance Break Down of HDGrid**

approaches the theoretical lower bound, reaching the maximum achievable compression efficiency. This observation confirms our theoretical analysis that lower-dimensional labels exhibit stronger prefix redundancy, thus allowing higher compression rates.

When the label dimension increases, the compression efficiency naturally decreases, yet it consistently remains above 50× across all datasets. Interestingly, when the label dimension is greater than one, reducing the number of sub-clusters $K_2$ further decreases the total memory footprint. Although more prefix-sum arrays need to be stored, each array becomes significantly more compressible. For example, on the `Deep1M` dataset, the total prefix-sum memory decreases from 3.7 GB to 1.3 GB when $K_2$ increases from 1 to 10. This empirically verifies **Theorem 3.1**, which predicts that finer-grained sub-clustering improves the overall compression efficiency.

*4.2.3 Performance Break-Down.* To analyze the performance impact of querying prefix-sum arrays and sub-cluster filtering in the two-level clustering structure, we perform a detailed breakdown of the execution time under Recall $\geq$ 90% across eight datasets. We divide the query pipeline into four stages: (1) prefix-sum array lookup for each sub-cluster, (2) filtering valid sub-clusters within selected top-level clusters, (3) fine-grained candidate filtering and search inside sub-clusters, and (4) final re-ranking. The results are shown in Figure 4.

Although HDGrid increases the number of clusters by 5–10× compared with the baseline, we observe that the time spent on cluster filtering remains comparable to—and often lower than—the time spent on searching within sub-clusters. Across all eight datasets, cluster filtering consistently accounts for a moderate portion of the total query time and never becomes a dominant bottleneck. This

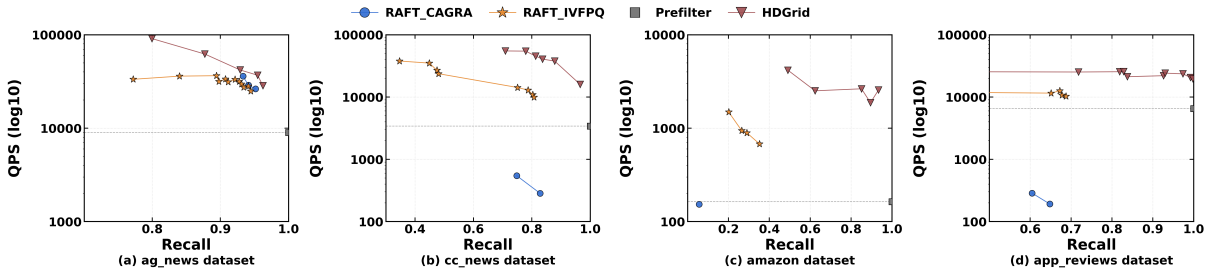


**Figure 5: Recall-QPS curves for real-world datasets**

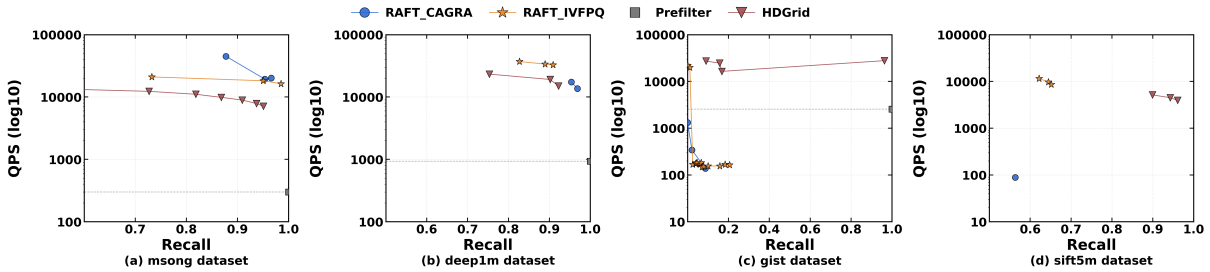


**Figure 6: Recall-QPS curves for synthetic datasets**

indicates that the additional hierarchy introduced by the two-level clustering, together with efficient prefix-sum based filtering, effectively offsets the potential overhead of managing more clusters.

An exception appears on the Amazon dataset, where the cluster filtering time slightly exceeds the in-cluster search time. This is because the dataset exhibits an extremely low filtering ratio: most irrelevant points are already pruned in the Morton-code stage, leaving a very small number of points for the search phase. As a result, the relative proportion of the filtering phase increases, though the absolute time remains low.

Overall, the breakdown results confirm that even after expanding the number of clusters by up to one order of magnitude, HDGrid maintains a balanced computation distribution. The efficient two-level filtering mechanism prevents the clustering phase from becoming a performance bottleneck, validating the scalability of our hierarchical design.

## 4.3 Comparison with Stitched Methods

We further compare HDGrid with GPU-based filtering baselines, including Pre-filtering, RAFT-IVFPQ + post-filtering, and CAGRA + post-filtering, to evaluate both efficiency and recall across varying filter selectivities and query complexities.

Across all datasets, HDGrid consistently outperforms the GPU pre-filtering baseline by 2×–10× in QPS, while maintaining comparable high recall. Although the pre-filtering approach achieves 100% recall—since filtering is applied strictly before vector retrieval—its performance is limited by the high cost of scanning and filtering large candidate sets, particularly under low selectivity workloads.

Compared with CAGRA + post-filtering and IVFPQ + post-filtering, HDGrid achieves consistently higher performance across datasets with filter selectivities less than or equal to 50%, and particularly excels in low- and variable-selectivity workloads. The only exception is the msong dataset, where all queries share a uniform 80% selectivity. In this case, both CAGRA and IVFPQ incur negligible post-filtering overhead, resulting in slightly better performance than HDGrid. Across all datasets, post-filtering methods suffer from significant degradation in both latency and recall, often failing to return more than 50% recall on datasets such as Amazon and GIST. HDGrid, on the other hand, maintains stable recall (>90%) and achieves up to 20× speedup over post-filtering baselines in the most complex workloads.

## 4.4 Comparison with Native Methods

We first compare HDGrid with state-of-the-art CPU-based supporting filtering indices, including iRangeGraph, ACORN, and the RFANN-dedicated variants (e.g., RangePQ-based and hybrid post-filtering implementations). All methods are evaluated under their optimal configurations to achieve at least 90% recall whenever possible.

Across all eight datasets, HDGrid consistently outperforms the best CPU-based baselines by 2–3 orders of magnitude in QPS at comparable recall levels (>90%). This improvement demonstrates the robustness of our two-stage filtering design to variations in filter selectivity and the number of filters.

The Amazon dataset (five range filters with highly variable per-dimension selectivity) and the GIST dataset (three range filters with overall selectivity below 0.1%) correspond to the cases with the lowest filter selectivities and the largest variation in query-level filtering ratios. For clarity, results with recall below 1% are omitted from the figures. In these settings, HDGrid achieves over 1,000× higher QPS than the best-performing CPU baseline while maintaining 90% recall.

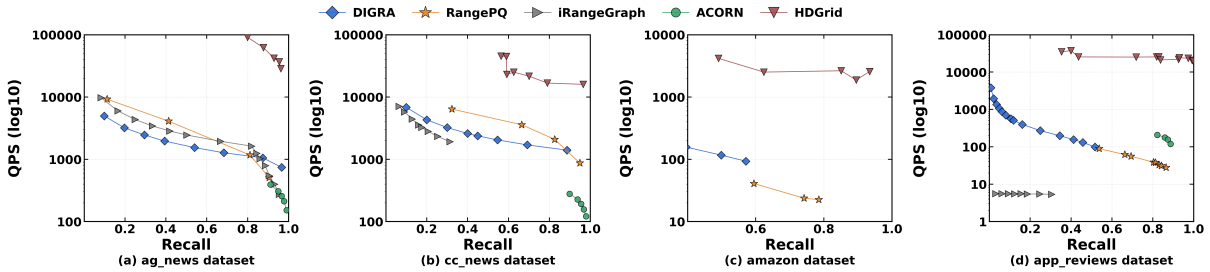


**Figure 7: Recall-QPS curves for real-world datasets**

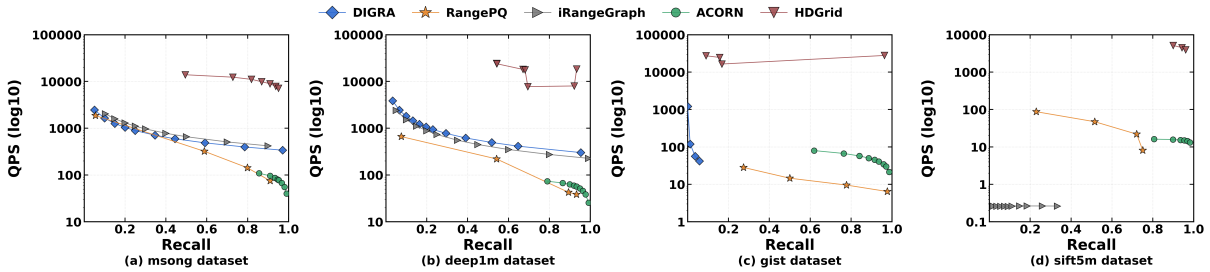


**Figure 8: Recall-QPS curves for synthetic datasets**

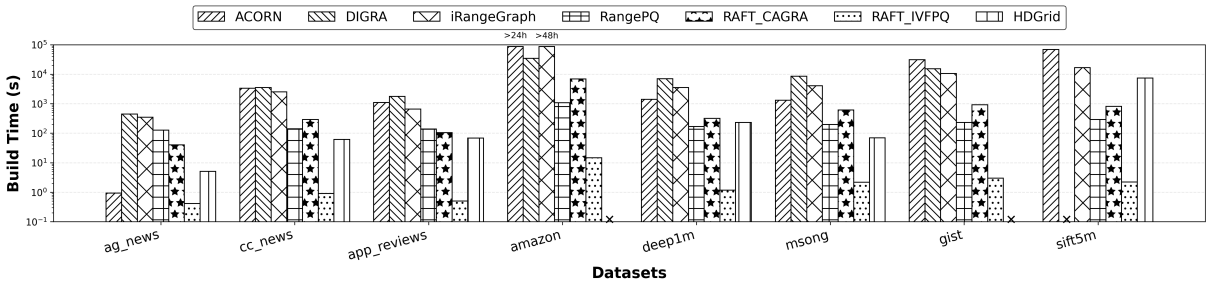


**Figure 9: Comparison of build time across different indexing methods and datasets.**

Notably, while some RFANN-dedicated indices and ACORN perform competitively on datasets with fixed and moderate filter ratios (e.g., ag_news, msong), their performance drops sharply when the filter selectivity varies significantly or when multiple filtering dimensions are applied.

In contrast, HDGrid maintains stable latency across workloads with filter selectivity ranging from 0.01% to 80% and with the number of filtering conditions varying from 1 to 5.

## 4.5 Indexing time comparison

In terms of index construction time, SuperGridPQ introduces additional preprocessing overhead for building filter-related structures, leading to a moderate increase compared with `RAFT-IVFPQ`. However, it still constructs significantly faster than CPU-based `RFANN`, achieving a 3–100× speedup, and in the `Amazon` dataset even more than four orders of magnitude faster. Compared with graph-based `CAGRA`, SuperGridPQ generally shows comparable or slightly faster build times. When compared with `RangePQ`, SuperGridPQ outperforms it on low-dimensional filtering workloads but can be slower on datasets with high-dimensional labels due to the larger number of prefix-sum arrays.

## 4.6 Memory consumption comparison

We first analyze the memory footprint of SuperGridPQ compared with state-of-the-art ANN indexes. As shown in Fig. 10, since SuperGridPQ builds several additional data structures beyond the inverted lists and PQ codebooks (e.g., prefix-sum arrays and Morton-code grids), its overall memory usage is naturally higher than that of the baseline `RAFT-IVFPQ`. However, compared with CPU-based native `RFANN` indexes, SuperGridPQ achieves a 3–10× reduction in index size, demonstrating that the additional filtering structures are still lightweight relative to CPU-based solutions. We also observe that graph-based RFANN indexes require substantially more memory, often exceeding 32 GB on most datasets, which makes them impractical to deploy directly on a single 32 GB V100 GPU.

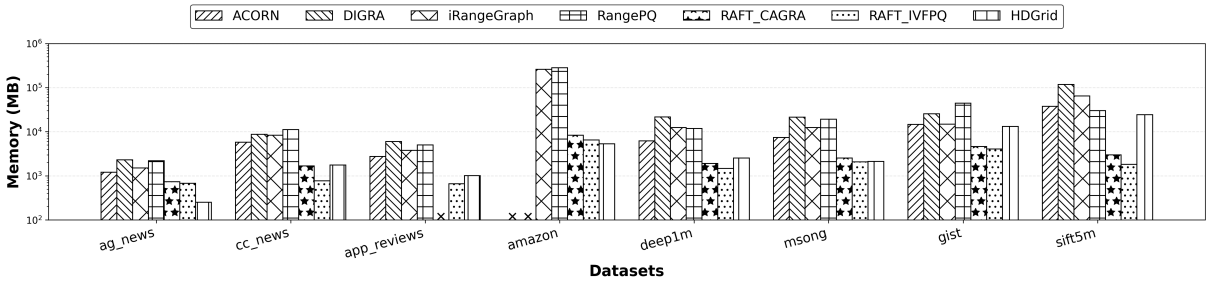


**Figure 10: Comparison of memory consumption across different indexing methods and datasets.**

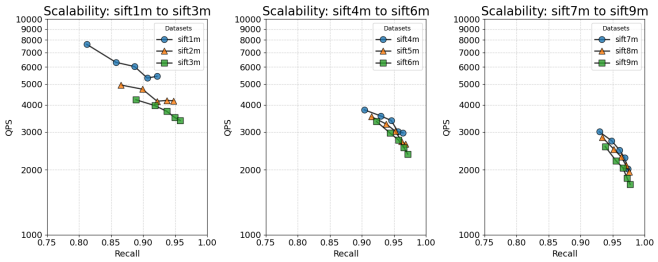


**Figure 11: Scalability Test of HDGrid**

Nevertheless, SuperGridPQ remains memory-efficient relative to CPU-based graph indexes. Campared with CAGRA, HDGrid and CAGRA exhibit complementary strengths. HDGrid requires less GPU memory on datasets with a single filtering condition, but as the filtering dimensionality increases, its memory consumption depends on the label distribution and can occasionally exceed that of CAGRA.

## 4.7 Scalability

To evaluate scalability, we conduct experiments on the `SIFT10M` dataset, which we partition into subsets ranging from 1M to 9M points. We measure the query throughput (QPS) at Recall $\geq$ 90% under a single filtering condition.

As shown in Figure 11, the QPS decreases only marginally as the dataset size increases from 1M to 9M, indicating that HDGrid exhibits near-linear scalability with respect to dataset size. This suggests that the hierarchical filtering and compressed prefix-sum design effectively control the search cost even when the index and candidate space grow substantially.

Overall, these results demonstrate that HDGrid achieves a favorable trade-off among memory efficiency, indexing overhead, and scalability. The index can be efficiently built and deployed on modern GPUs while maintaining stable query performance as the dataset grows.

## 5 Conclusion

We proposed HDGrid, the first native high-dimensional grid index for Range-filtering Approximate Nearest Neighbor search. By partitioning the high-dimensional filter space into regular grid cells and leveraging Morton code–based spatial ordering, HDGrid minimizes irrelevant candidate accesses while preserving high recall.By partitioning the high-dimensional filter space into regular grid cells and leveraging Morton code–based spatial ordering, HDGrid minimizes irrelevant candidate accesses while preserving high recall. Moreover, HDGrid introduces a compressed grid structure that significantly reduces memory footprint and maximizes GPU throughput. Extensive experiments on diverse datasets from BigVectorBench demonstrate that HDGrid achieves state-of-the-art accuracy while outperforming CPU-based baselines by up to four orders of magnitude in query throughput. Our compression algorithm further enables full in-GPU deployment, eliminating memory bottlenecks and maximizing parallelism.

## References

[1] 2016. Weaviate. https://github.com/weaviate/weaviate
[2] 2016. Yahoo.Nearest neighbor search with neighborhood graph and tree for high-dimensional data. https://github.com/yahoojapan/NGT
[3] 2018. RAFT. https://github.com/rapidsai/raft
[4] Pankaj K Agarwal. 2017. Range searching. In *Handbook of discrete and computational geometry*. Chapman and Hall/CRC, 1057–1092.
[5] Jon Louis Bentley. 1975. Multidimensional binary search trees used for associative searching. *Commun. ACM* 18, 9 (1975), 509–517.
[6] Guy E Blelloch. 1990. Prefix sums and their applications. (1990).
[7] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The Faiss library. (2024). arXiv:2401.08281 [cs.LG]
[8] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. 2024. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130* (2024).
[9] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2013. Optimized product quantization. *IEEE transactions on pattern analysis and machine intelligence* 36, 4 (2013), 744–755.
[10] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2013. Optimized product quantization for approximate nearest neighbor search. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2946–2953.
[11] Antonin Guttman. 1984. R-trees: A dynamic index structure for spatial searching. In *Proceedings of the 1984 ACM SIGMOD international conference on Management of data*. 47–57.
[12] Xiaoxin He, Yijun Tian, Yifei Sun, Nitesh Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, and Bryan Hooi. 2024. G-retriever: Retrieval-augmented generation for textual graph understanding and question answering. *Advances in Neural Information Processing Systems* 37 (2024), 132876–132907.
[13] Joseph M Hellerstein, Jeffrey F Naughton, and Avi Pfeffer. 1995. *Generalized search trees for database systems*. September.
[14] Herve Jegou, Matthijs Douze, and Cordelia Schmid. 2010. Product quantization for nearest neighbor search. *IEEE transactions on pattern analysis and machine intelligence* 33, 1 (2010), 117–128.
[15] Mengxu Jiang, Zhi Yang, Fangyuan Zhang, Guanhao Hou, Jieming Shi, Wenchao Zhou, Feifei Li, and Sibo Wang. 2025. DIGRA: A Dynamic Graph Indexing for Approximate Nearest Neighbor Search with Range Filter. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–26.
[16] Bernal Jimenez Gutierrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. Hipporag: Neurobiologically inspired long-term memory for large language

models. *Advances in Neural Information Processing Systems* 37 (2024), 59532–59569.

[17] Anqi Liang, Pengcheng Zhang, Bin Yao, Zhongpu Chen, Yitong Song, and Guangxu Cheng. 2024. UNIFY: Unified Index for Range Filtered Approximate Nearest Neighbors Search. *arXiv preprint arXiv:2412.02448* (2024).

[18] James B McQueen. 1967. Some methods of classification and analysis of multivariate observations. In *Proc. of 5th Berkeley Symposium on Math. Stat. and Prob.* 281–297.

[19] Yitong Meng, Xinyan Dai, Xiao Yan, James Cheng, Weiwen Liu, Jun Guo, Benben Liao, and Guangyong Chen. 2020. Pmd: An optimal transportation-based user distance for recommender systems. In *European Conference on Information Retrieval*. Springer, 272–280.

[20] Jason Mohoney, Anil Pacaci, Shihabur Rahman Chowdhury, Ali Mousavi, Ihab F Ilyas, Umar Farooq Minhas, Jeffrey Pound, and Theodoros Rekatsinas. 2023. High-throughput vector similarity search in knowledge graphs. *Proceedings of the ACM on Management of Data* 1, 2 (2023), 1–25.

[21] Shumpei Okura, Yukihiro Tagami, Shingo Ono, and Akira Tajima. 2017. Embedding-based news recommendation for millions of users. In *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*. 1933–1942.

[22] Hiroyuki Ootomo, Akira Naruse, Corey Nolet, Ray Wang, Tamas Feher, and Yong Wang. 2024. Cagra: Highly parallel graph construction and approximate nearest neighbor search for gpus. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 4236–4247.

[23] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. Acorn: Performant and predicate-agnostic search over vector embeddings and structured data. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–27.

[24] Arkadiusz Paterek. 2007. Improving regularized singular value decomposition for collaborative filtering. In *Proceedings of KDD cup and workshop*, Vol. 2007. 5–8.

[25] Frank Ramsak, Volker Markl, Robert Fenk, Martin Zirkel, Klaus Elhardt, and Rudolf Bayer. 2000. Integrating the UB-tree into a database system kernel.. In *VLDB*, Vol. 2000. 263–272.

[26] Shubhabrata Sengupta, Mark Harris, Yao Zhang, and John D Owens. 2007. Scan primitives for GPU computing. (2007).

[27] Kento Sugiura and Yoshiharu Ishikawa. 2023. Z-ordered Range Refinement for Multi-dimensional Range Queries. *arXiv preprint arXiv:2305.12732* (2023).

[28] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, et al. 2021. Milvus: A purpose-built vector data management system. In *Proceedings of the 2021 International Conference on Management of Data*. 2614–2627.

[29] Chuangxian Wei, Bin Wu, Sheng Wang, Renjie Lou, Chaoqun Zhan, Feifei Li, and Yuanzhe Cai. 2020. Analyticdb-v: A hybrid analytical engine towards query fusion for structured and unstructured data. *Proceedings of the VLDB Endowment* 13, 12 (2020), 3152–3165.

[30] Orion Weller, Michael Boratko, Iftekhar Naim, and Jinhyuk Lee. 2025. On the theoretical limitations of embedding-based retrieval. *arXiv preprint arXiv:2508.21038* (2025).

[31] Yuexuan Xu, Jianyang Gao, Yutong Gou, Cheng Long, and Christian S Jensen. 2024. irangegraph: Improvising range-dedicated graphs for range-filtering nearest neighbor search. *Proceedings of the ACM on Management of Data* 2, 6 (2024), 1–26.

[32] Fangyuan Zhang, Mengxu Jiang, Guanhao Hou, Jieming Shi, Hua Fan, Wenchao Zhou, Feifei Li, and Sibo Wang. 2025. Efficient Dynamic Indexing for Range Filtered Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–26.

[33] Qianxi Zhang, Shuotao Xu, Qi Chen, Guoxin Sui, Jiadong Xie, Zhizhen Cai, Yaoqi Chen, Yinxuan He, Yuqing Yang, Fan Yang, et al. 2023. {VBASE}: Unifying online vector similarity search and relational queries via relaxed monotonicity. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. 377–395.

[34] Chaoji Zuo, Miao Qiao, Wenchao Zhou, Feifei Li, and Dong Deng. 2024. SeRF: segment graph for range-filtering approximate nearest neighbor search. *Proceedings of the ACM on Management of Data* 2, 1 (2024), 1–26.

# Appendix

## A Proof of lemmas and theories

### A.1 Lemma3.1

Proof. The total number of points in the dataset is $N$. These points are partitioned into $K_1K_2$ disjoint sub-clusters, $C_1, C_2, \ldots$, such that $N = \sum_i |C_i|$.

The array $P_i$ is a sequence of prefix sums based on the counts of points of $C_i$ falling into fixed grid regions. Since the counts are non-negative, the values in $P_i$ are non-decreasing integers.

The maximum value in the prefix sum array $P_i$ is the final element, which represents the total count of all points of $C_i$ across all grid regions. Therefore,

$$\max(P_i) = |C_i|$$

Since the values in $P_i$ are non-negative integers that range from 0 up to a maximum of $|C_i|$, the number of distinct values in the array, $card(P_i)$, can be at most $|C_i| + 1$ (if the array contains 0 and all integers up to $|C_i|$). A conservative upper bound that is sufficient for our argument is:

$$card(P_i) \leq |C_i|$$

Now, summing this inequality over all $K_1K_2$ sub-clusters:

$$\sum_{i=1}^{K_1K_2} card(P_i) \leq \sum_{i=1}^{K_1K_2} |C_i|$$

Since the sub-clusters $C_i$ are disjoint and partition the dataset $\mathcal{U}$, their total size is $N$:

$$\sum_{i=1}^{K_1K_2} |C_i| = N$$

Substituting this total back into the inequality yields the result:

$$\sum_{i=1}^{K_1K_2} card(P_i) \leq N$$

Intuitively, $card(P_i)$ measures the complexity of cluster $C_i$'s distribution relative to the grid structure. The total measure of complexity across all clusters is bounded by the total number of points, $N$, because the number of distinct counts in $P_i$ can be no greater than the total number of points belonging to $C_i$. □

### A.2 Theory1

Proof. **Lower bound.** By definition of the compressed prefix sum array, each non-compressible substructure requires at least one storage unit. Therefore, the space lower bound $\Theta\left(\frac{G\,K_1\,K_2}{S[l]}\right)$ directly follows from the structural definition and is omitted here for brevity.

**Upper bound.** Let $R$ denote the number of rows in the compressed array. According to the hierarchical and block organization of the data structure, the number of rows satisfies

$$R = \Theta\left(\frac{G}{S[l]}\right).$$

Consider an arbitrary row, and let $S_{\text{row}}$ denote the total number of *sumNode* elements stored in this row. By **Lemma 3.2** and by the definition of the compressed prefix sum array (which explicitly avoids storing duplicate or redundant prefix values), each original data item can contribute to at most a constant number of distinct *sumNode*s within a single row. Hence, there exists a constant $c > 0$ such that

$$S_{\text{row}} \leq c \cdot N,$$

implying that each row occupies $O(N)$ space. In the general case, the row must still encode the cumulative contribution of all $N$ original elements, leading to an asymptotically tight bound

$$S_{\text{row}} = \Theta(N).$$

Combining the per-row complexity with the number of rows yields the overall upper bound:

$$\text{Space} = R \cdot S_{\text{row}} = \Theta\left(\frac{G}{S[l]}\right) \cdot \Theta(N) = \Theta\left(\frac{N\,G}{S[l]}\right).$$

Together with the lower bound derived from the definition, this completes the proof. □

### A.3 Theory2

Proof. **Setup and notation.** Let $R$ denote the number of rows in the compressed array at layer $l$. By the hierarchical/block organisation of the structure we have

$$R \;=\; \Theta\left(\frac{G}{S[l]}\right) \;=\; \Theta\left(\frac{G}{\sqrt{G}}\right) \;=\; \Theta(\sqrt{G}).$$

From the previous theorem (upper bound with general $S[l]$) we know that the worst-case space per stored row is $\Theta(N)$; hence the worst-case total storage (without accounting for row deduplication) is $\Theta(N) \cdot R = \Theta(N\sqrt{G})$. We next show that, in expectation, row-level deduplication and the mandatory-first-entry constraint reduce the expected stored fraction of rows by a factor bounded above by the stated $\max(\cdot,\cdot)$ term, yielding the claimed expected bound.

**Probability that a row is identical to its predecessor.** Consider two consecutive logical rows in the structure. If the later row is *entirely zero* (i.e. all its $G$ positions are zero after projection onto the considered row's support), then its prefix-sum vector equals that of the previous row and therefore it need not be stored explicitly. Hence a sufficient condition for a row to be identical to its predecessor is that the row is all-zero.

Let $M$ denote the number of non-zero positions (or an available upper bound thereof) in the global array; sampling $s = \sqrt{G}$ positions per row (as in the layer/block arrangement) yields the exact combinatorial probability that a given row is entirely zero:

$$p_{\text{zero}} \;=\; \frac{\binom{G-M}{s}}{\binom{G}{s}} \;=\; \prod_{i=0}^{s-1} \frac{G-M-i}{G-i}.$$

Using the standard inequality $1 - x \leq e^{-x}$ and $s = \sqrt{G}$, we obtain the exponential upper bound

$$p_{\text{zero}} \;\leq\; \exp\Big(-\frac{sM}{G}\Big) \;=\; \exp\Big(-\frac{M}{\sqrt{G}}\Big).$$

Recalling the relation between $M$ and $N$ used in the earlier lemma (or setting $M$ to the problem-specific upper bound $N/(C\,S_c)$), we further bound

$$p_{\text{zero}} \;\leq\; \exp\Big(-\frac{N}{C\,S_c\,\sqrt{G}}\Big).$$

Thus the probability that a row is *not* all-zero (and hence, under the sufficient condition considered, could require explicit storage) is at most $1 - p_{\text{zero}}$.

**Lower bound on the fraction of rows that must be stored.** The previous paragraph gives a probabilistic reason for row equality via the all-zero event, but it does not capture deterministic constraints that force storage even when rows are not all-zero. In particular, each secondary table (secondary index or per-block subordinate structure) must store its first entry; these mandatory entries impose a non-vanishing lower bound on the fraction of rows that cannot be eliminated by deduplication. Concretely, standard combinatorial/entropy arguments on the distribution of first entries across $M$ possible values imply that at least a fraction on the order of $M^{-1/(2(l-1))}$ of rows (up to constants) must be distinct in expectation (see main text for the per-secondary-table occupancy argument). We therefore obtain a deterministic-type lower bound $f_{\min}$ on the expected stored fraction of rows:

$$f_{\min} \;\geq\; M^{-\frac{1}{2(l-1)}}.$$

**Combining the two effects.** Let $X$ be the number of rows actually stored (random variable). For $i \geq 2$ row $i$ is stored only if it is not identical to row $i-1$ (and the first row is stored according to the mandatory-first-entry rule). Taking expectations, using independence assumptions between rows as in the earlier lemma (rows are formed from independent projections and their all-zero events are independent), and combining the probabilistic and deterministic constraints above, we obtain

$$\mathbb{E}[X] \;\leq\; 1 + (R-1)\cdot\max\big(1 - p_{\text{zero}},\, f_{\min}\big).$$

Multiplying by the per-row storage cost $\Theta(N)$ yields

$$\mathbb{E}[\text{Space}] \;=\; \Theta(N)\cdot\mathbb{E}[X] \;=\; \Theta(N)\left(1 + (R-1)\max\Big(1 - p_{\text{zero}},\, f_{\min}\Big)\right).$$

Substituting $R = \Theta(\sqrt{G})$ and discarding lower-order additive constants gives

$$\mathbb{E}[\text{Space}] \;=\; \Theta\Big(N\sqrt{G}\Big)\cdot\max\Big(1 - p_{\text{zero}},\, M^{-\frac{1}{2(l-1)}}\Big).$$

**Replace $1 - p_{\text{zero}}$ by its exponential bound.** Using the exponential bound for $p_{\text{zero}}$ derived above we have $1 - p_{\text{zero}} \leq 1 - \exp(-N/(C\,S_c\sqrt{G}))$. For the regime of interest (large parameters typical in our applications) we can upper-bound $1 - p_{\text{zero}}$ by a constant multiple of $\exp(-N/(C\,S_c\sqrt{G}))$ (this is the standard simplification when the exponential term is small). Thus

$$\mathbb{E}[\text{Space}] \;=\; O\left(N\sqrt{G}\cdot\max\left(1 - \exp\Big(-\frac{N}{CS_c\sqrt{G}}\Big),\; M^{-\frac{1}{2(l-1)}}\right)\right).$$

**Why the little-o notation.** The stated bound is written with $o(\cdot)$ (rather than $\Theta(\cdot)$) because the first row (and possibly a small constant number of initial rows) contains strictly fewer *sumNode*s than a typical later row; consequently, when $R > 1$ the per-row $\Theta(N)$ worst-case is not attained uniformly across all rows, and the asymptotic leading constant is strictly smaller than that of the worst-case product $N\sqrt{G}$. Therefore the expected total storage lies in $o\big(N\sqrt{G}\cdot\max(\cdot,\cdot)\big)$ rather than $\Theta(\cdot)$. Combining the above estimates completes the proof. □

## A.4 Theory3

Proof. Recall that a single JumpIn call is invoked to traverse one contiguous interval (segment) of the underlying one-dimensional ordering (Morton/Z-order) that intersects the query filter. Hence the total number of JumpIn calls for a given range query is upper-bounded by the number of disjoint contiguous Morton-intervals into which the query region is decomposed.

It is a classic result that encoding a range (rectangular) query using Morton order yields $\Theta(\mathcal{A}(\mathcal{R}))$ such contiguous segments, where $\mathcal{A}(\mathcal{R})$ is the surface area of the query region (see e.g. [? ? ]). We therefore denote by $m = \Theta(\mathcal{A}(\mathcal{R}))$ the number of contiguous Morton segments generated for the query when no additional splits are applied.

Now consider the effect of applying $s$ splits to the data structure. Each split can at most increase the total number of contiguous segments intersecting the query by one: a split that falls inside a previously contiguous segment may split that segment into two, thereby adding at most one extra segment; a split that falls outside the query or exactly on a segment boundary does not increase the number of segments. Hence after applying $s$ splits the total number of segments is at most $m + s$. Consequently, the number of JumpIn calls is upper-bounded by

$$m + s \;=\; \Theta\big(\mathcal{A}(\mathcal{R})\big) + s \;=\; O\big(\mathcal{A}(\mathcal{R}) + s\big).$$

*Conclusion.* Putting the two bounds together, we obtain

$$\#\text{JumpIn} \;=\; \Theta\big(\mathcal{A}(\mathcal{R}) + s\big),$$

as required. □